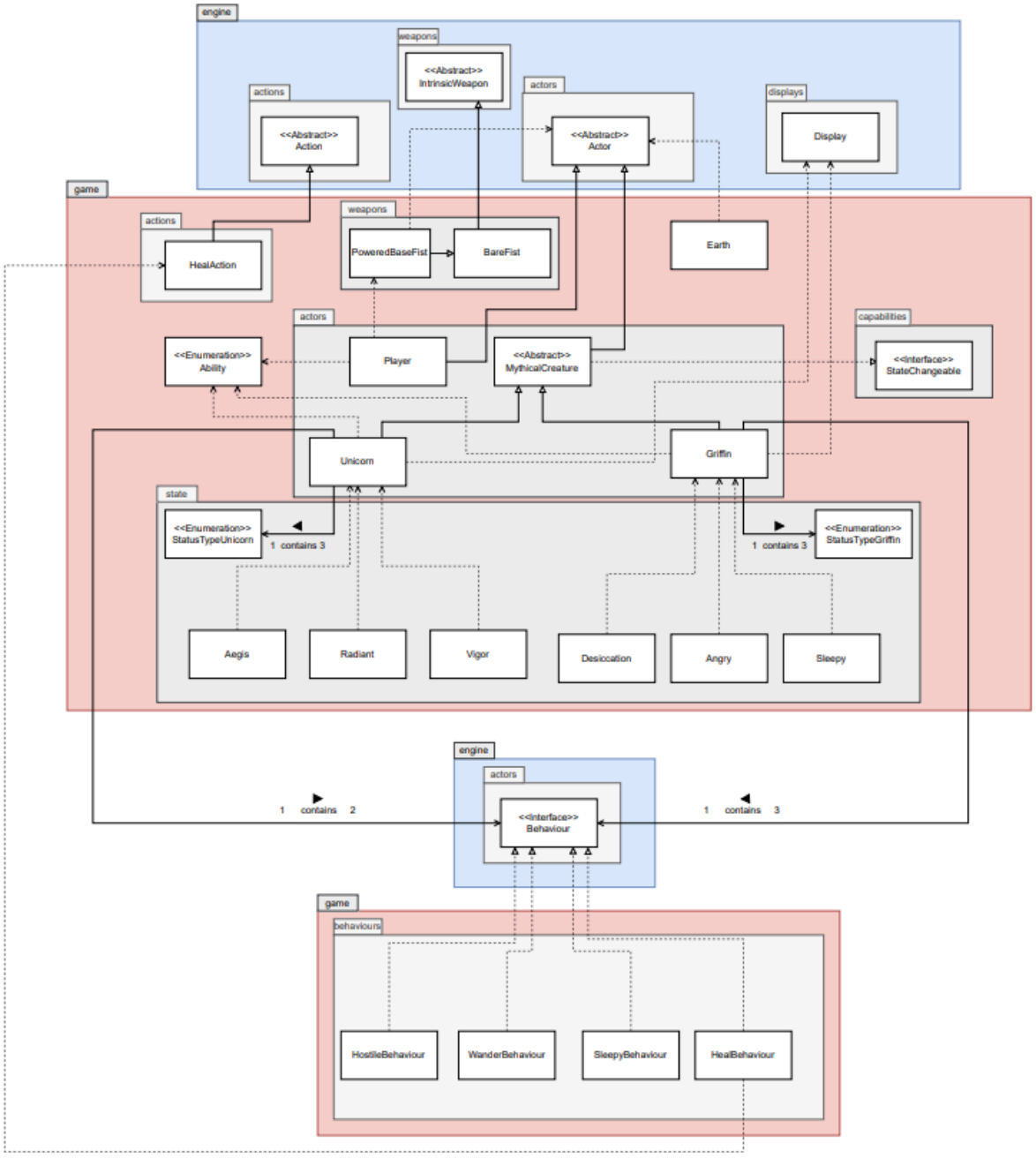
REQ 5 UML Class Diagram:



Design Rationale

REQ 5

A. Implementation of new creatures Unicorn and Griffin
Design 1: Both Unicorn and Griffin classes are created by directly extending the existing Actor class.

Pros	Cons
Simple because it is straightforward to implement with minimal overhead.	Duplication risk as shared behaviors or attributes between mythical creatures would need to be re-implemented in each class.
Direct access as both creatures can directly use all functionalities provided by the Actor class without additional abstraction layers.	Have lower scalability because adding more mythical creatures later may result in repetitive code.
Less initial code because there is no need to introduce a new superclass.	There will be weaker abstraction because it does not clearly represent the conceptual grouping of mythical creatures.

Design 2: An abstract class MythicalCreatures extends the Actor class. Both Unicorn and Griffin then extend MythicalCreatures.

Pros	Cons
Code can be reused if there are common behaviors, abilities, or attributes that can be centralized in the abstract class.	Slightly more complex because it introduces an extra inheritance layer, which adds complexity compared to direct extension.
Provides more scalability because it is easier to add more mythical creatures in the future by extending the abstract class.	
Improves abstraction as it encapsulates the concept of mythical creatures, making the design more meaningful and maintainable.	
Improves flexibility as shared methods can be declared abstract, forcing subclasses to provide their own implementations where necessary.	

B. Changing states of Griffin and Unicorn
Design 1: When state changes are required, the code checks the type of the creature using instanceof and then applies downcasting to execute the appropriate state-changing logic.

Pros	Cons
Simple to implement because it is straightforward logic without introducing additional abstractions.	Poor scalability when there are more creatures and states are introduced, the number of instanceof checks grows, leading to bloated and hard-to-maintain code.
A temporary quick solution works well if there are only a few creatures or limited state-change cases.	Code violates OOP principles due to excessive use of 'instanceof' and downcasting is seen as a code smell because it often suggests a breach of the Open-Closed Principle
Easier direct control because each state change is explicitly handled in one place.	Weaker abstraction as it does not clearly represent the conceptual grouping of mythical creatures.

Design 2: An interface StateChangeable defines a method (e.g., changeState()) that all state-changing creatures must implement. The MythicalCreature abstract class will implement this interface, and subclasses like *Griffin* and *Unicorn* will provide their own logic for state changes.

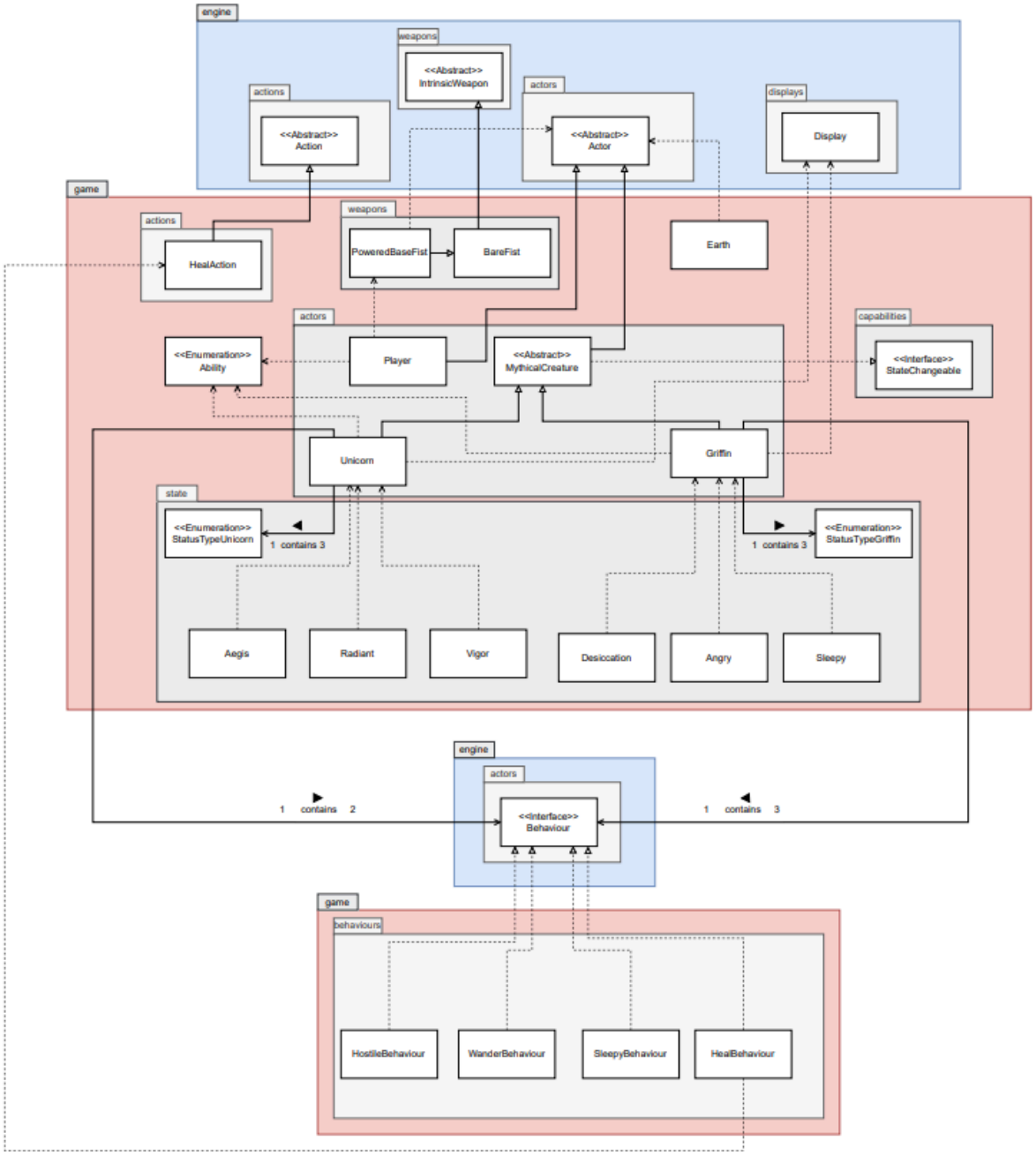
Pros	Cons
Promotes encapsulation as each creature handles its own state logic internally.	Might have slightly higher complexity as it requires designing and maintaining an additional interface.
Better scalability since it is easy to add new creatures with state-change capabilities without modifying existing code.	
Promotes flexibility because different creatures can implement unique state-change rules while sharing a common contract.	
Cleaner design since it avoids repetitive instanceof checks and makes the design more extensible.	

C:
Design 1: states are represented using constants, booleans, or string values directly within the creature classes (e.g., "ANGRY").

Pros	Cons
Simple implementation because it is easy to set up without defining additional types.	Error-prone are likely due to typo (e.g., "ANGRY" vs "angry") and harder to validate at compile time.
Fewer class files which keeps everything contained within the creature classes.	Poor readability because strings are scattered across the codebase hence reducing clarity.
Flexibility as developers can represent states in any form without being restricted to an enum definition.	Difficult maintenance: Adding/removing states requires updating several places in the code.

Design 2: States are represented using enumeration

Pros	Cons
Strong typing because it ensures only valid states can be used, with compile-time safety.	Extra abstraction: Introduces additional files/types in the codebase.
Improve maintainability because adding or modifying states only requires changes in the enum definition.	Less flexible for dynamic states: Enums are fixed at compile time, making them less suited for states that may change dynamically at runtime.
Promotes consistency as it standardizes the way of managing states across different mythical creatures.	



The diagram above shows the final design of requirement 5, which utilizes Design 2 of parts A,B,C from above for Unicorn and Griffin implementation, Design 2 from the state-change design, and the use of enumerations for state management. This design implements the *Unicorn* and *Griffin* classes that extend from the abstract MythicalCreature class, ensuring shared traits are abstracted for code reuse. Both classes manage their states through a StateChangeable interface and state-specific enumerations (StatusTypeGriffin, StatusTypeUnicorn).

By introducing the MythicalCreature abstract class, we reduce code duplication and achieve the DRY principle, since common methods and attributes are centralized rather than reimplemented in every creature class. This design also adheres to the Open-Closed

Principle of SOLID, as new mythical creatures can be added easily by extending the abstract class without modifying existing code.

The StateChangeable interface ensures that each creature encapsulates its own state-changing logic, rather than relying on instanceof checks. This follows the Single Responsibility Principle, where each class is responsible only for its own behavior, and the Dependency Inversion Principle, since higher-level code depends on interfaces (StateChangeable) rather than concrete implementations. Using different enums for each creature's states makes the code easier to read and maintain, while also applying the Interface Segregation Principle, as only creatures with states implement state-related behavior.

Finally, adopting enumerations to represent states provides strong typing and clarity, while avoiding error-prone string comparisons. This keeps the design simple, robust, and in line with the KISS principle, as developers can easily identify and manage possible states without unnecessary complexity. Overall, the combination of abstraction, interfaces, and enums results in a design that is maintainable, extensible, and aligned with core software engineering principles.